

Лабораторная работа №5

Имитационное моделирование

Чистов Даниил Максимович

Содержание

1	Цель работы	4
2	Задание	5
3	О сетях Петри	6
3.1	Общая информация	6
3.2	Правило срабатывания перехода	7
4	Задача «Обедающие философы»	8
4.1	Классическая постановка	8
4.2	Проблема взаимной блокировки (deadlock)	9
5	Выполнение лабораторной работы	10
5.1	Создание анимации динамики работы сети Петри	26
5.2	Создание итогового отчёта	27
5.3	Столбцы для состояния «Ест»	28
6	Выводы	30

Список иллюстраций

5.1	Инициализация проекта	10
5.2	Создание модели DiningPhilosophers.jl	11
5.3	Создание скрипта dining_philosophers.jl	21
5.4	Результат работы dining_philosophers.jl	23
5.5	Результат работы классической модели	24
5.6	Результат работы модели с арбитром	25
5.7	Итоговый отчёт	29

```
import Pkg
Pkg.activate("../project")
Pkg.instantiate()
```

```
Activating project at `C:\Users\12232\Documents\GitHub\2026-
1--study--simulation-modeling\labs\lab05\project`
```

1 Цель работы

Целью данной работы ознакомление с сетями Петри на примере задачи «Обедающие философы».

2 Задание

1. Ознакомиться с сетями Петри
2. Ознакомиться с задачей «Обедающие философы»
3. Реализовать решение проблемы deadlock, возникающей при базовой реализации решения задачи

3 О сетях Петри

3.1 Общая информация

Сеть Петри есть математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники).

- Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия).
- Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов.
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы.
- Они могут срабатывать, изменяя состояние модели.
- Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот.
- Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели.
- Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

3.2 Правило срабатывания перехода

Переход считается разрешённым (enabled), если каждая из его входных позиций содержит как минимум столько фишек, сколько указывает кратность входной дуги. Разрешённый переход может сработать (fire). Этот процесс:

- Удаляет из каждой входной позиции количество фишек, равное кратности входной дуги.
- Добавляет в каждую выходную позицию количество фишек, равное кратности выходной дуги.

Срабатывание перехода есть атомарная операция, которая происходит мгновенно и меняет маркировку сети. Если разрешено несколько переходов, любой из них может сработать.

4 Задача «Обедающие философы»

Данная задача наглядно демонстрирует явления взаимной блокировки (deadlock) и голодания (starvation) при конкурентном доступе к разделяемым ресурсам. Это одна из самых известных задач, демонстрирующая проблемы параллелизма, в частности, взаимные блокировки (deadlocks) и состояние гонки (race conditions).

4.1 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов

Правила поведения философов:

- Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест.
- Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа.
- Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся.
- Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

4.2 Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть взаимная блокировка.

Для предотвращения deadlock предложены различные подходы:

- Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно.
- Асимметричное правило — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот.
- Атомарный захват — философ берёт обе вилки одновременно (если они свободны) иначе не берёт ни одной.
- Использование мьютексов и условных переменных в программных реализациях.

Задача «Обедающие философы» является классическим примером для моделирования с помощью сетей Петри. Каждый философ и каждая вилка представляются позициями, а переходы соответствуют действиям: «взять левую вилку», «взять правую вилку», «начать есть», «положить вилки». Сеть Петри наглядно показывает возникновение deadlock и позволяет экспериментировать с различными механизмами синхронизации, например, введением арбитра.

5 Выполнение лабораторной работы

Перед выполнением работы, поставим следующие цели:

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

В папке lab05 инициализирую проект для данной лабораторной работы (рис. 5.1)

```
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab05> ls

Каталог: C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab05

Mode                LastWriteTime         Length Name
----                -
d-----          17.04.2026         0:07      presentation
d-----          18.04.2026        16:06      project
d-----          17.04.2026         0:07      report
```

Рисунок 5.1: Инициализация проекта

В папке /src создаю файл DiningPhilosophers.jl. Это код модели задачи обедающих философов, который будет использоваться другими скриптами (рис. 5.2)

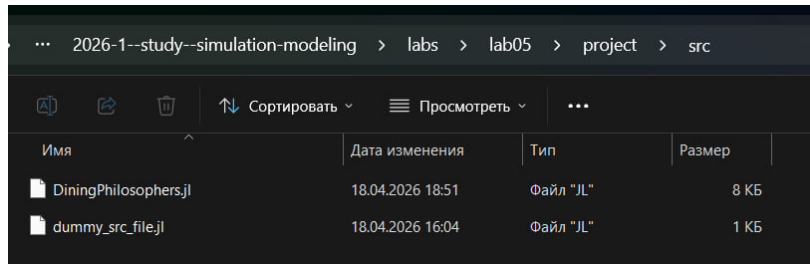


Рисунок 5.2: Создание модели DiningPhilosophers.jl

Ниже представлен код данной модели:

```
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution
```

5.0.1 Определение простой структуры PetriNet

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end
```

```

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
)
    incidence = zeros(Int, n_places, n_transitions)
    if isempty(place_names)
        place_names = [Symbol("p$i") for i = 1:n_places]
    end
    if isempty(transition_names)
        transition_names = [Symbol("t$i") for i = 1:n_transitions]
    end
    PetriNet(n_places, n_transitions, incidence, place_names, transition_names)
end

function add_arc!(net::PetriNet, place::Int, transition::Int, sign::Int)
    net.incidence[place, transition] += sign
end

```

5.0.2 Построение сетей Петри

```

function build_classical_network(N::Int)
    n_places = 4N
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N

```

```

    net.place_names[i] = Symbol("Think_$i")
    net.place_names[N+i] = Symbol("Hungry_$i")
    net.place_names[2N+i] = Symbol("Eat_$i")
    net.place_names[3N+i] = Symbol("Fork_$i")
end

for i = 1:N
    net.transition_names[i] = Symbol("GetLeft_$i")
    net.transition_names[N+i] = Symbol("GetRight_$i")
    net.transition_names[2N+i] = Symbol("PutForks_$i")
end

for i = 1:N
    think = i
    hungry = N + i
    eat = 2N + i
    left_fork = 3N + i
    right_fork = 3N + (i % N + 1)

    get_left = i
    get_right = N + i
    put_forks = 2N + i

    add_arc!(net, think, get_left, -1)
    add_arc!(net, left_fork, get_left, -1)
    add_arc!(net, hungry, get_left, +1)

    add_arc!(net, hungry, get_right, -1)
    add_arc!(net, right_fork, get_right, -1)

```

```

        add_arc!(net, eat, get_right, +1)

        add_arc!(net, eat, put_forks, -1)
        add_arc!(net, think, put_forks, +1)
        add_arc!(net, left_fork, put_forks, +1)
        add_arc!(net, right_fork, put_forks, +1)
    end

    u0 = zeros(Float64, n_places)
    for i = 1:N
        u0[i] = 1.0                # Think_i
        u0[3N+i] = 1.0            # Fork_i
    end
    return net, u0, net.place_names
end

function build_arbiter_network(N::Int)
    n_places = 4N + 1
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N
        net.place_names[i] = Symbol("Think_$i")
        net.place_names[N+i] = Symbol("Hungry_$i")
        net.place_names[2N+i] = Symbol("Eat_$i")
        net.place_names[3N+i] = Symbol("Fork_$i")
    end
    net.place_names[4N+1] = :Arbiter
end

```

```

for i = 1:N
    net.transition_names[i] = Symbol("GetLeft_$i")
    net.transition_names[N+i] = Symbol("GetRight_$i")
    net.transition_names[2N+i] = Symbol("PutForks_$i")
end

arbiter_idx = 4N + 1

for i = 1:N
    think = i
    hungry = N + i
    eat = 2N + i
    left_fork = 3N + i
    right_fork = 3N + (i % N + 1)

    get_left = i
    get_right = N + i
    put_forks = 2N + i

    add_arc!(net, think, get_left, -1)
    add_arc!(net, left_fork, get_left, -1)
    add_arc!(net, arbiter_idx, get_left, -1)
    add_arc!(net, hungry, get_left, +1)

    add_arc!(net, hungry, get_right, -1)
    add_arc!(net, right_fork, get_right, -1)
    add_arc!(net, eat, get_right, +1)

```

```

        add_arc!(net, eat, put_forks, -1)
        add_arc!(net, think, put_forks, +1)
        add_arc!(net, left_fork, put_forks, +1)
        add_arc!(net, right_fork, put_forks, +1)
        add_arc!(net, arbiter_idx, put_forks, +1)
    end

    u0 = zeros(Float64, n_places)
    for i = 1:N
        u0[i] = 1.0
        u0[3N+i] = 1.0
    end
    u0[arbiter_idx] = N - 1
    return net, u0, net.place_names
end

```

5.0.3 Детерминированное моделирование (ODE)

```

function vectorfield(net::PetriNet, rates = ones(net.n_transitions))
    function f!(du, u, params, t)
        a = zeros(net.n_transitions)
        for j = 1:net.n_transitions
            rate = rates[j]
            prod = rate
            for i = 1:net.n_places
                if net.incidence[i, j] < 0
                    prod *= u[i] ^ (-net.incidence[i, j])
                end
            end
        end
    end
end

```



```

        end
        a[j] = prod
    end
    du .= net.incidence * a
end
return f!
end

function simulate_ode(net::PetriNet, u0::Vector{Float64}, tmax::Float64)
    f = vectorfield(net)
    prob = ODEProblem(f, u0, (0.0, tmax))
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    for i = 1:net.n_places
        df[!, String(net.place_names[i])] = sol[i, :]
    end
    return df
end
end

```

5.0.4 Стохастическое моделирование (алгоритм Гиллеспи)

```

function simulate_stochastic(
    net::PetriNet,
    u0::Vector{Float64},
    tmax::Float64;
    rates = ones(net.n_transitions),
    rng = Random.GLOBAL_RNG,
)

```

```

u = copy(u0)
t = 0.0
times = [t]
states = [copy(u)]

while t < tmax
    a = zeros(net.n_transitions)
    for j = 1:net.n_transitions
        rate = rates[j]
        prod = rate
        for i = 1:net.n_places
            if net.incidence[i, j] < 0
                prod *= u[i] ^ (-net.incidence[i, j])
            end
        end
        a[j] = prod
    end
    a0 = sum(a)
    if a0 == 0
        break
    end
    dt = -log(rand(rng)) / a0
    r = rand(rng) * a0
    cumsum = 0.0
    chosen = 1
    for j = 1:net.n_transitions
        cumsum += a[j]
        if r <= cumsum

```

```

        chosen = j
        break
    end
end
end
for i = 1:net.n_places
    u[i] += net.incidence[i, chosen]
end
t += dt
if t <= tmax
    push!(times, t)
    push!(states, copy(u))
end
end
df = DataFrame(time = times)
for i = 1:net.n_places
    df[!, String(net.place_names[i])] = [s[i] for s in states]
end
return df
end
end

```

5.0.5 Обнаружение deadlock

```

function detect_deadlock(df::DataFrame, net::PetriNet; tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])] for i = 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 && u_last[i] < -net.incidence[i, j]
                can_fire = false
            end
        end
        if can_fire
            u_last = u_last + net.incidence[:, j]
        end
    end
end

```

```

        can_fire = false
        break
    end
end
if can_fire
    return false
end
end
return true
end

```

5.0.6 Визуализация

```

function plot_marking_evolution(df::DataFrame, N::Int)
    plots = []
    for group in ["Think", "Hungry", "Eat", "Fork"]
        p = plot(xlabel = "Time", ylabel = group, title = "$group s
        for i = 1:N
            col = "$(group)_$i"
            if col in names(df)
                plot!(df.time, df[:, col], label = "$(group)_$i")
            end
        end
        push!(plots, p)
    end
    return plot(plots..., layout = (4, 1), size = (800, 1000))
end

```

В папке /scripts создаю файл dining_philosophers.jl (рис. 5.3).

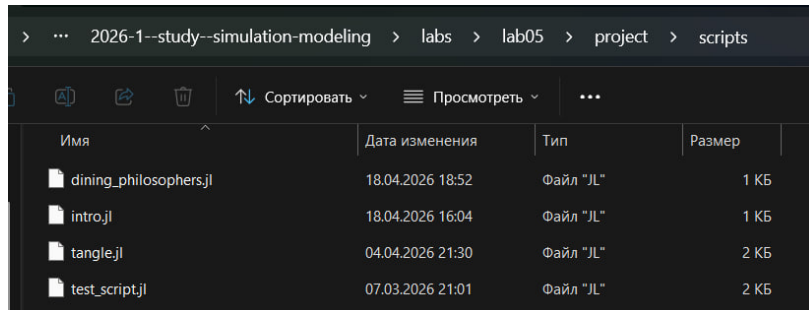


Рисунок 5.3: Создание скрипта dining_philosophers.jl

Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:

- классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
- модифицированная модель с арбитром, которая должна предотвращать deadlock.

Классическая сеть должна приводить к deadlock (через некоторое время все философы оказываются в состоянии Hungry, ни один не может есть). Если detect_deadlock возвращает true, это подтверждает теоретический вывод: классическая постановка задачи обедающих философов ведёт к взаимной блокировке.

Сеть с арбитром (дополнительная позиция с N-1 фишками) должна избегать deadlock: detect_deadlock возвращает false. Это демонстрирует один из способов синхронизации для предотвращения тупиков.

Ниже приведён код dining_philosophers.jl:

```

using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

println("=== XXXXXXXXXXXX XXXX (XXX XXXXXXXX) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock XXXXXXXXXX: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

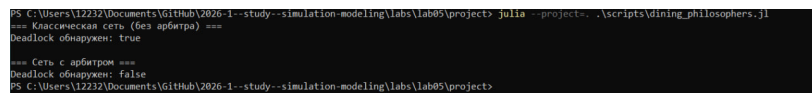
println("\n=== XXXXX X XXXXXXXXX ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock XXXXXXXXXX: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))

```

Данный код: * Создаёт классическую сеть Петри для N=5 философов с помощью build_classical_network. * Запускает стохастическую симуляцию (алгоритм Гиллеспи)

на время $t_{max} = 50.0$. * Сохраняет полную историю маркировок в CSV-файл (dining_classic.csv). * Анализирует последнее состояние на предмет deadlock с помощью detect_deadlock и выводит результат в консоль. * Строит графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как classic_simulation.png. * Повторяет шаги 1–5 для сети с арбитром (build_arbiter_network), сохраняя результаты в dining_arbiter.csv и arbiter_simulation.png

Код успешно работает (рис. 5.4).



```
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab05\projects> julia --project=. \scripts\dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true
=== Сеть с арбитром ===
Deadlock обнаружен: false
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab05\projects>
```

Рисунок 5.4: Результат работы dining_philosophers.jl

В результате работы скрипта были получены следующие графики:

- Результат работы классической модели (рис. 5.5).
- Результат работы модели с арбитром (рис. 5.6).

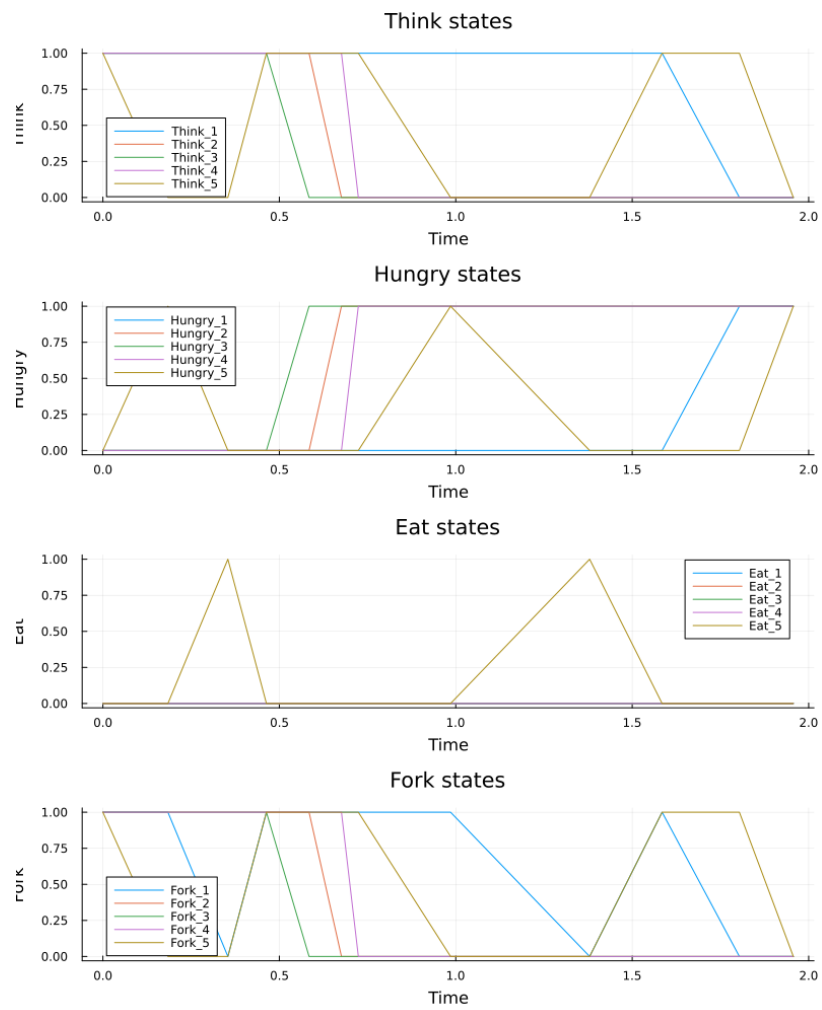


Рисунок 5.5: Результат работы классической модели

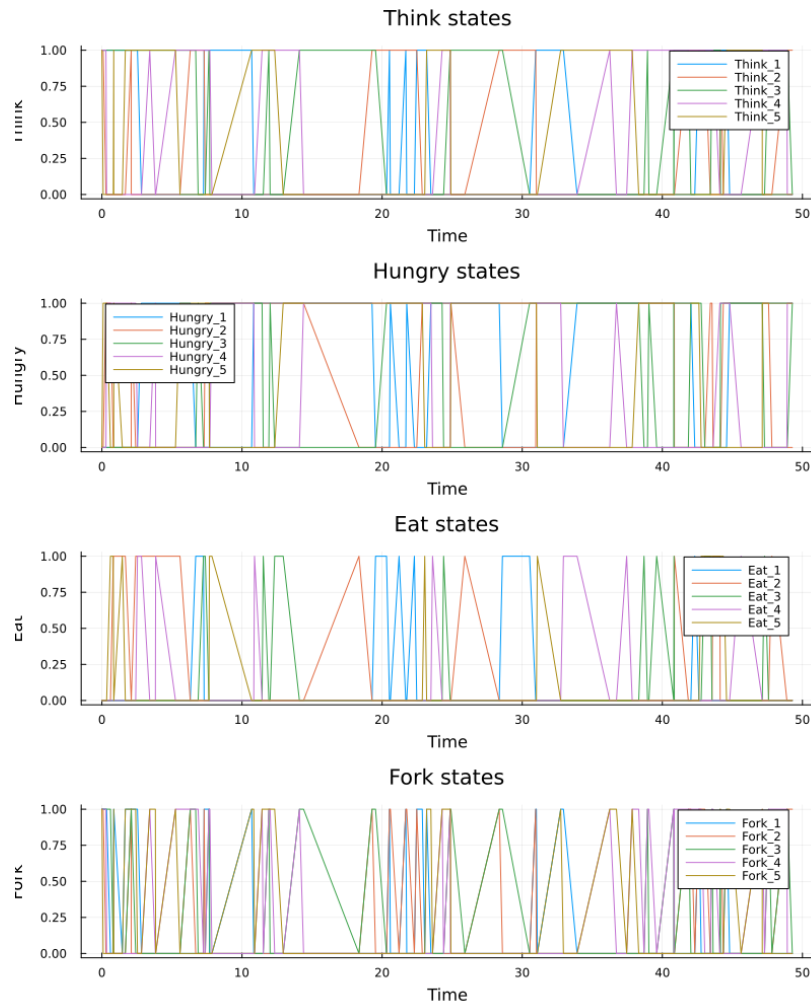


Рисунок 5.6: Результат работы модели с арбитром

Графики эволюции маркировки позволяют визуально увидеть, как меняются состояния.

- В классической сети рано или поздно все Eat_i становятся нулевыми, а $Hungry_i$ – единичными (или наоборот).
- В сети с арбитром наблюдается постоянное чередование приёма пищи

5.1 Создание анимации динамики работы сети

Петри

Для наглядной демонстрации динамики работы сети Петри во времени создадим скрипт `dining_philosophers_animation.jl`.

```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "XXXXXX",
        ylabel = "XXXX",
        title = "XXXX = $(round(row.time, digits=2))",
    )
end
```

```

    )
    xticks!(1:length(u), string.(names), rotation = 45)
end

gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("XXXXXXXX XXXXXXXX X plots/philosophers_simulation.gif")

```

Данный скрипт: * Берёт классическую сеть (для малого числа философов ($N=3$) для упрощения визуализации). * Запускает стохастическую симуляцию на $t_{\max} = 30.0$. * Создаёт анимацию с помощью макроса **animate**, где каждый кадр – это столбчатая диаграмма текущей маркировки (количество фишек в каждой позиции). * Сохраняет анимацию как GIF-файл philosophers_simulation.gif с частотой 5 кадров в секунду.

С результатом работы скрипта вы можете ознакомиться в папке lab05/report/images или lab05/project/plots - там находится файл анимации philosophers_simulation.gif.

5.2 Создание итогового отчёта

Для сравнительного анализа двух моделей требуется создать скрипт dining_philosophers_report.jl, который будет загружать ранее сохранённые CSV-файлы (dining_classic.csv и dining_arbiter.csv) из папки data/, извлекать столбцы, соответствующие состоянию Eat_i для каждого философа, строить два временных графика (один под другим): динамику «едящих» философов для классической сети и для сети с арбитром, затем сохранять объединённый график как final_report.png в папку plots/.

```

using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5

```

5.3 Столбцы для состояния «Ест»

```

eat_cols = [Symbol("Eat_<math>i</math>") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = ["<math>x_i</math>" for i = 1:N],
    xlabel = "<math>x_1, x_2, x_3, x_4</math>",
    ylabel = "<math>t(1/0)</math>",
    title = "<math>x_1, x_2, x_3, x_4</math> <math>t(1/0)</math>",
)

p2 = plot(
    df_arbiter.time,
    Matrix(df_arbiter[:, eat_cols]),
    label = ["<math>x_i</math>" for i = 1:N],
    xlabel = "<math>x_1, x_2, x_3, x_4</math>",
    ylabel = "<math>t(1/0)</math>",
    title = "<math>t(1/0)</math> <math>x_1, x_2, x_3, x_4</math>",
)

```

```

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))

println("XXXXX XXXXXXXX X plots/final_report.png")

```

В результате был получен следующий график (рис. 5.7).

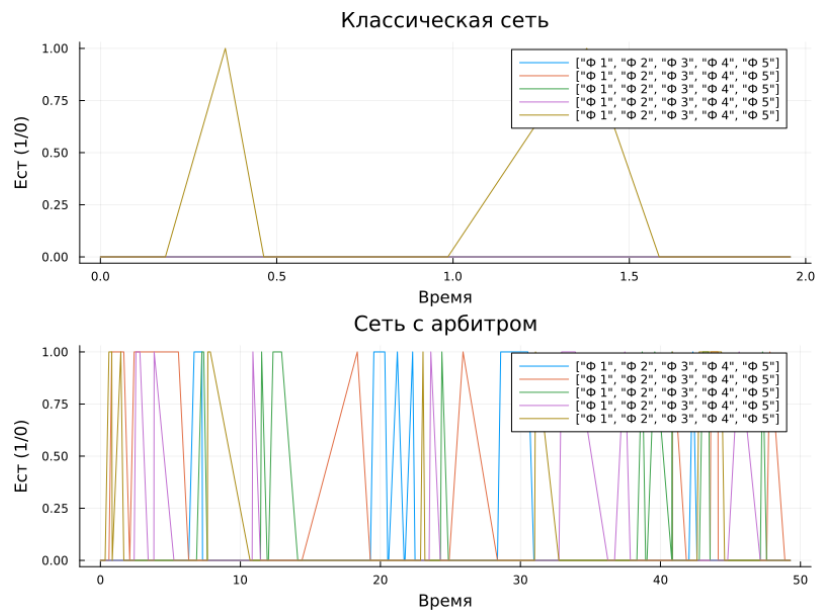


Рисунок 5.7: Итоговый отчёт

Изучив график, можно сделать следующие выводы:

- На графике классической сети видно, что через некоторое время все линии Eat_i падают до нуля и остаются на нуле - это и есть deadlock, философы больше не могут есть.
- На графике сети с арбитром линии Eat_i колеблются, всегда есть хотя бы один философ, который ест (или они поочерёдно получают доступ).
- Отсутствие длительных нулевых периодов подтверждает, что система жива и не блокируется.

По данному графику можно сделать вывод, что введение арбитра является эффективным решением проблемы deadlock

6 Выводы

В результате выполнения данной лабораторной работы я изучил сети Петри и поработал с задачей «Обедающие философы», решив проблему deadlock путём введения арбитра.